

```

%{                                     // SECCION 1 Declaraciones de C-Yacc

#include <stdio.h>
#include <ctype.h>                     // declaraciones para tolower
#include <string.h>                    // declaraciones para cadenas
#include <stdlib.h>                    // declaraciones para exit ()

#define FF fflush(stdout);            // para forzar la impresion inmediata

int yylex () ;
int yyerror (char *mensaje) ;
char *my_malloc (int) ;
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;

char temp [2048] ;
int output_pos = 0;

typedef struct s_local_var {
    char *name;
    struct s_local_var *next;
} t_local_var;

t_local_var *local_var_table = NULL;
char current_function[256] = "";

```

```

void add_local_var(char *name) {
    t_local_var *new_var = (t_local_var *) my_malloc(sizeof(t_local_var));
    new_var->name = gen_code(name);
    new_var->next = local_var_table;
    local_var_table = new_var;
}

int is_local_var(char *name) {
    t_local_var *current = local_var_table;
    while (current != NULL) {
        if (strcmp(current->name, name) == 0) {
            return 1;
        }
        current = current->next;
    }
    return 0;
}

void clear_local_vars() {
    local_var_table = NULL;
}

char *concat_with_function(char *var_name) {
    if (is_local_var(var_name)){
        sprintf(temp, "%s_%s", current_function, var_name);
        return gen_code(temp);
    } else{
        return var_name;
    }
}

```

```
    }  
    ;  
}  
  
typedef struct s_attr {  
    int value ;  
    char *code ;  
} t_attr ;  
  
#define YYSTYPE t_attr  
  
%}  
  
%token NUMBER  
%token IDENTIF  
%token INTEGER  
%token STRING  
%token MAIN  
%token WHILE  
%token IF  
%token ELSE  
%token FOR  
%token RETURN  
%token PRINTF  
%token PUTS  
%token SWITCH  
%token CASE  
%token DEFAULT
```

```
%token BREAK
```

```
%token INC
```

```
%token DEC
```

```
%token LOGICAL_AND
```

```
%token LOGICAL_OR
```

```
%token EQ
```

```
%token NE
```

```
%token LE
```

```
%token GE
```

```
%right '='
```

```
%left LOGICAL_OR
```

```
%left LOGICAL_AND
```

```
%left EQ NE
```

```
%left '<' '>' LE GE
```

```
%left '+' '-'
```

```
%left '*' '/' '%'
```

```
%right UNARY_SIGN
```

```
%right '!'
```

```
%right ELSE
```

```
%%
```

```
programa:      declaraciones_globales      { printf("%s", $1.code); }
               definiciones_funciones      { ; }
               ;
```

```

declaraciones_globales:                                { sprintf(temp, ""); $$code = gen_code(temp); }
    | declaracion_global
      declaraciones_globales                          { sprintf(temp, "%s%s", $1.code, $2.code);
                                                         $$code = gen_code(temp); }

    ;

declaracion_global: INTEGER lista_global ';'          { $$code = $2.code; }

    ;

lista_global: init_global                             { $$code = $1.code; }
    | init_global ',' lista_global                  { sprintf(temp, "%s%s", $1.code, $3.code);
                                                         $$code = gen_code(temp); }

    ;

init_global: IDENTIF                                 { sprintf(temp, "(setq %s 0)\n", $1.code);
                                                         $$code = gen_code(temp); }
    | IDENTIF '=' NUMBER                           { sprintf(temp, "(setq %s %d)\n", $1.code, $3.value);
                                                         $$code = gen_code(temp); }
    | IDENTIF '[' NUMBER ']'                       { sprintf(temp, "(setq %s (make-array %d))\n", $1.code,
$3.value);
                                                         $$code = gen_code(temp); }

    ;

definiciones_funciones: definicion_funcion
    definiciones_funciones { ; }

    |
    { ; }

    ;

```

```

definicion_funcion: MAIN                                { strcpy(current_function, "main");
                                                         printf("(defun main (")); }

    r_definicion_main                                    { ; }
    | IDENTIF                                           { strcpy(current_function, $1.code);
                                                         printf("(defun %s", $1.code); }

    r_definicion_resto                                  { ; }
    ;

r_definicion_main: '(' ')' bloque                        { printf("\n %s\n", $3.code);
                                                         clear_local_vars(); }
    ;

r_definicion_resto: '(' parametros ')' bloque           { printf(" (%s) %s\n", $2.code, $4.code);
                                                         clear_local_vars(); }
    ;

parametros:                                              { $$code = gen_code(""); }
    | lista_parametros                                  { $$code = $1.code; }
    ;

lista_parametros: INTEGER IDENTIF                        { add_local_var($2.code);
                                                         char * var_name = concat_with_function($2.code);
                                                         sprintf(temp, "%s", var_name);
                                                         $$code = gen_code(temp); }
    | INTEGER IDENTIF ',' lista_parametros             { add_local_var($2.code);
                                                         char * var_name = concat_with_function($2.code);

```

```

                                sprintf(temp, "%s %s", var_name, $4.code);
                                $$code = gen_code(temp); }

;

bloque: '{' lista_sentencias '}'
        { $$code = $2.code; }
        ;

lista_sentencias:
        | sentencia lista_sentencias
        { $$code = gen_code(""); }
        { sprintf(temp, "%s%s", $1.code, $2.code);
          $$code = gen_code(temp); }
        ;

sentencia: declaracion_local
        | asignacion ';'
        | print_funcion ';'
        | while_stmt
        | if_stmt
        | for_stmt
        | switch_stmt
        | llamada_funcion ';'
        | bloque
        | return_stmt ';'
        | BREAK ';'
        { $$code = gen_code(""); }
        ;

declaracion_local: INTEGER lista_local ';'
        { $$code = $2.code; }
        ;

```

```

lista_local: init_local                                { $$code = $1.code; }
              | init_local',' lista_local              { sprintf(temp, "%s%s", $1.code, $3.code);
                                                         $$code = gen_code(temp); }

              ;

init_local: IDENTIF                                    { add_local_var($1.code);
                                                         sprintf(temp, "(setq %s%s 0) ", current_function, $1.code);
                                                         $$code = gen_code(temp); }

              | IDENTIF '=' NUMBER                    { add_local_var($1.code);
                                                         sprintf(temp, "(setq %s%s %d) ", current_function, $1.code,
$3.value);

                                                         $$code = gen_code(temp); }

              | IDENTIF '[' NUMBER ']'                { add_local_var($1.code);
                                                         sprintf(temp, "(setq %s%s (make-array %d)) ",
current_function, $1.code, $3.value);

                                                         $$code = gen_code(temp); }

              ;

asignacion: IDENTIF '=' expresion                      { char *var_name = concat_with_function($1.code);
                                                         sprintf(temp, "(setf %s %s) \n", var_name, $3.code);
                                                         $$code = gen_code(temp); }

              | IDENTIF '[' expresion ']' '='          { char *var_name = concat_with_function($1.code);
                                                         sprintf(temp, "(setf (aref %s %s) %s) ", var_name, $3.code,
$6.code);

                                                         $$code = gen_code(temp); }

              ;

```

```

expresion: expr_logica                { $$ = $1; }
        ;

expr_logica: expr_comparacion          { $$ = $1; }
        | expr_logica LOGICAL_AND
          expr_comparacion             { sprintf(temp, "(and %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_logica LOGICAL_OR
          expr_comparacion             { sprintf(temp, "(or %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        ;

expr_comparacion: expr_aditiva          { $$ = $1; }
        | expr_aditiva EQ expr_aditiva { sprintf(temp, "(= %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_aditiva NE expr_aditiva { sprintf(temp, "(/= %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_aditiva '<' expr_aditiva { sprintf(temp, "< %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_aditiva '>' expr_aditiva { sprintf(temp, "> %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_aditiva LE expr_aditiva { sprintf(temp, "(<= %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        | expr_aditiva GE expr_aditiva { sprintf(temp, "(>= %s %s)", $1.code, $3.code);
                                        $$code = gen_code(temp); }
        ;

expr_aditiva: expr_multiplicativa      { $$ = $1; }

```

```

| expr_aditiva '+'
  expr_multiplicativa
    { sprintf(temp, "(+ %s %s)", $1.code, $3.code);
      $$code = gen_code(temp); }

| expr_aditiva '-'
  expr_multiplicativa
    { sprintf(temp, "(- %s %s)", $1.code, $3.code);
      $$code = gen_code(temp); }

;

expr_multiplicativa: expr_unaria
    { $$ = $1; }
  | expr_multiplicativa '*'
    expr_unaria
      { sprintf(temp, "(* %s %s)", $1.code, $3.code);
        $$code = gen_code(temp); }
  | expr_multiplicativa '/'
    expr_unaria
      { sprintf(temp, "(/ %s %s)", $1.code, $3.code);
        $$code = gen_code(temp); }
  | expr_multiplicativa '%'
    expr_unaria
      { sprintf(temp, "(mod %s %s)", $1.code, $3.code);
        $$code = gen_code(temp); }

;

expr_unaria: expr_primaria
    { $$ = $1; }
  | '+' expr_unaria %prec UNARY_SIGN { $$ = $2; }
  | '-' expr_unaria %prec UNARY_SIGN { sprintf(temp, "(- %s)", $2.code);
                                       $$code = gen_code(temp); }
  | '!' expr_unaria
    { sprintf(temp, "(not %s)", $2.code);
      $$code = gen_code(temp); }

;

```

```

expr_primaria: IDENTIF { sprintf(temp, "%s", concat_with_function($1.code));
                        $$code = gen_code(temp);
                        }
    | NUMBER { sprintf(temp, "%d", $1.value);
              $$code = gen_code(temp); }
    | '(' expresion ')' { $$code = $2.code; }
    | IDENTIF '[' expresion ']' { char *var_name = concat_with_function($1.code);
                                sprintf(temp, "(aref %s %s)", var_name, $3.code);
                                $$code = gen_code(temp); }
    | IDENTIF '(' argumentos_llamada ')' { sprintf(temp, "(%s %s)", $1.code, $3.code);
                                          $$code = gen_code(temp); }
    ;

printf_funcion: printf_funcion { ; }
    | puts_funcion { ; }
    ;

printf_funcion: PRINTF '(' STRING ')' { sprintf(temp, "(print \"%s\")", $3.code);
                                         $$code = gen_code(temp); }
    | PRINTF '(' STRING ','
      argumentos_printf ')' { sprintf(temp, "%s", $5.code);
                              $$code = gen_code(temp); }
    ;

argumentos_printf: argumento_printf { $$code = $1.code; }
    | argumento_printf ','
      argumentos_printf { sprintf(temp, "%s %s", $1.code, $3.code);
                          $$code = gen_code(temp); }

```

```

;

argumento_printf: expresion          { sprintf(temp, "(princ %s)", $1.code);
                                     $$code = gen_code(temp); }
                                     | STRING
                                     { sprintf(temp, "(princ \"%s\")", $1.code);
                                     $$code = gen_code(temp); }
                                     ;

puts_funcion: PUTS '(' STRING ')'    { sprintf(temp, "(print \"%s\") \n", $3.code);
                                     $$code = gen_code(temp); }
                                     | PUTS '(' expresion ')'
                                     { sprintf(temp, "(print %s) \n", $3.code);
                                     $$code = gen_code(temp); }
                                     ;

while_stmt: WHILE '(' expresion ')' bloque { sprintf(temp, "(loop while %s do\n %s)", $3.code, $5.code);
                                             $$code = gen_code(temp); }
                                             ;

if_stmt: IF '(' expresion ')' bloque %prec ELSE { sprintf(temp, "(if %s (progn %s)) \n", $3.code, $5.code);
                                                  $$code = gen_code(temp); }
          | IF '(' expresion ')' bloque
            ELSE bloque
          { sprintf(temp, "(if %s (progn %s) (progn %s)) \n", $3.code,
$5.code, $7.code);
          $$code = gen_code(temp); }
          ;

for_stmt: FOR '(' for_inicial ';' for_condicion ';'

```

```

        for_incremento ')' bloque
$5.code, $9.code, $7.code);

        { sprintf(temp, "%s\n(loop while %s do %s %s \n) ", $3.code,
        $$code = gen_code(temp); }

        ;

for_inicial: asignacion
        |
        ;

        { $$ = $1; }
        { $$code = gen_code(""); }

for_condicion: expresion
        |
        ;

        { $$ = $1; }
        { $$code = gen_code("1"); }

for_incremento: inc_dec
        |
        ;

        { $$ = $1; }
        { $$code = gen_code(""); }

inc_dec: IDENTIF '=' IDENTIF '+' NUMBER
$5.value);

        { char *var_name = concat_with_function($1.code);
        sprintf(temp, "(setf %s (+ %s %d))", var_name, var_name,
        $$code = gen_code(temp); }

        | IDENTIF '=' IDENTIF '-' NUMBER
$5.value);

        { char *var_name = concat_with_function($1.code);
        sprintf(temp, "(setf %s (- %s %d))", var_name, var_name,
        $$code = gen_code(temp); }

        | INC '(' IDENTIF ')'
$3.code);
        sprintf(temp, "(setf %s (+ %s 1))", var_name, var_name);
        $$code = gen_code(temp); }

```

```

| DEC '(' IDENTIF ')' { char *var_name = concat_with_function($3.code);
                        sprintf(temp, "(setf %s (- %s 1))", var_name, var_name);
                        $$code = gen_code(temp); }

;

switch_stmt: SWITCH '(' expresion ')' switch_bloque { sprintf(temp, "(case %s\n%s)", $3.code, $5.code);
                                                    $$code = gen_code(temp); }

switch_bloque: '{' lista_case default_case '}' { sprintf(temp, "%s%s", $2.code, $3.code);
                                                $$code = gen_code(temp); };

lista_case:
| case_item lista_case { $$code = gen_code(""); }
{ sprintf(temp, "%s\n%s", $1.code, $2.code);
  $$code = gen_code(temp); };

case_item: CASE expresion ':' lista_sentencias { sprintf(temp, " (%s %s)", $2.code, $4.code);
                                                $$code = gen_code(temp); };

default_case:
| DEFAULT ':' lista_sentencias { $$code = gen_code(""); }
{ sprintf(temp, " (otherwise %s)", $3.code);
  $$code = gen_code(temp); };

return_stmt: RETURN expresion { sprintf(temp, "(return-from %s %s) ", current_function,
$2.code);

| RETURN { sprintf(temp, "(return-from %s) ", current_function);
          $$code = gen_code(temp); }

;

```

```

llamada_funcion: IDENTIF '(' argumentos_llamada ')' { sprintf(temp, "(%s %s) \n", $1.code, $3.code);
                                                    $$code = gen_code(temp); }

;

argumentos_llamada:
    | lista_argumentos
    ;

lista_argumentos: expresion
    | expresion ',' lista_argumentos
    ;
    { $$code = $1.code; }
    { sprintf(temp, "%s %s", $1.code, $3.code);
      $$code = gen_code(temp); }

```

```

%%                                     // SECCION 4Codigo en C

```

```

int n_line = 1 ;

```

```

int yyerror (char *mensaje)
{
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
    return 0;
}

```

```

char *int_to_string (int n)
{
    char ltemp [2048] ;
    sprintf (ltemp, "%d", n) ;
}

```

```

    return gen_code (ltemp) ;
}

char *char_to_string (char c)
{
    char ltemp [2048] ;
    sprintf (ltemp, "%c", c) ;
    return gen_code (ltemp) ;
}

char *my_malloc (int nbytes)
{
    char *p ;
    static long int nb = 0 ;
    static int nv = 0 ;

    p = malloc (nbytes) ;
    if (p == NULL) {
        fprintf (stderr, "No queda memoria para %d bytes mas\n", nbytes) ;
        fprintf (stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv) ;
        exit (0) ;
    }
    nb += (long) nbytes ;
    nv++ ;
    return p ;
}

typedef struct s_keyword {

```

```
char *name ;
int token ;
} t_keyword ;

t_keyword keywords [] = {
    "main",      MAIN,
    "int",        INTEGER,
    "while",      WHILE,
    "if",          IF,
    "else",        ELSE,
    "for",         FOR,
    "return",      RETURN,
    "printf",      PRINTF,
    "puts",        PUTS,
    "switch",      SWITCH,
    "case",        CASE,
    "default",     DEFAULT,
    "break",       BREAK,
    "&&",          LOGICAL_AND,
    "||",          LOGICAL_OR,
    "==",          EQ,
    "!=",          NE,
    "<=",          LE,
    ">=",          GE,
    "inc",         INC,
    "dec",         DEC,
    NULL,          0
} ;
```

```
t_keyword *search_keyword (char *symbol_name)
{
    int i = 0 ;
    while (keywords[i].name != NULL) {
        if (strcmp(keywords[i].name, symbol_name) == 0) {
            return &keywords[i] ;
        }
        i++ ;
    }
    return NULL ;
}
```

```
char *gen_code (char *name)
{
    char *p ;
    int l ;
    l = strlen (name)+1 ;
    p = (char *) my_malloc (l) ;
    strcpy (p, name) ;
    return p ;
}
```

```
int yylex ()
{
    // NO MODIFICAR ESTA FUNCION SIN PERMISO
    int i ;
    unsigned char c ;
```

```
unsigned char cc ;
char ops_expandibles [] = "!<=|>%&/+-*" ;
char temp_str [256] ;
t_keyword *symbol ;

do {
    c = getchar () ;

    if (c == '#') { // Ignora las lineas que empiezan por #  (#define, #include)
        do {          // OJO que puede funcionar mal si una linea contiene #
            c = getchar () ;
        } while (c != '\n') ;
    }

    if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario
        cc = getchar () ;
        if (cc != '/') { // Si el siguiente char es / es un comentario, pero...
            ungetc (cc, stdin) ;
        } else {
            c = getchar () ; // ...
            if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea
                do {          // Se trata de codigo inline (Codigo embebido en C)
                    c = getchar () ;
                    putchar (c) ;
                } while (c != '\n') ;
            } else {          // ==> comentario, ignorar la linea
                while (c != '\n') {
                    c = getchar () ;
                }
            }
        }
    }
}
```

```

        }
    }
}
} else if (c == '\\') c = getchar () ;

if (c == '\n')
    n_line++ ;

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

if (c == '\"') {
    i = 0 ;
    do {
        c = getchar () ;
        temp_str [i++] = c ;
    } while (c != '\"' && i < 255) ;
    if (i == 256) {
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
    }
    // habria que leer hasta el siguiente " , pero, y si falta?
    temp_str [--i] = '\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;
    scanf ("%d", &yylval.value) ;
//    printf ("\nDEV: NUMBER %d\n", yylval.value) ;           // PARA DEPURAR

```

```

        return NUMBER ;
    }

    if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
        i = 0 ;
        while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
            (c >= '0' && c <= '9') || c == '_') && i < 255) {
            temp_str [i++] = tolower (c) ;
            c = getchar () ;
        }
        temp_str [i] = '\0' ;
        ungetc (c, stdin) ;

        yylval.code = gen_code (temp_str) ;
        symbol = search_keyword (yylval.code) ;
        if (symbol == NULL) { // no es palabra reservada -> identificador antes vrvariable
//            printf ("\nDEV: IDENTIF %s\n", yylval.code) ; // PARA DEPURAR
            return (IDENTIF) ;
        } else {
//            printf ("\nDEV: OTRO %s\n", yylval.code) ; // PARA DEPURAR
            return (symbol->token) ;
        }
    }

    if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles
        cc = getchar () ;
        sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
        symbol = search_keyword (temp_str) ;
    }

```

```
        if (symbol == NULL) {
            ungetc (cc, stdin) ;
            yylval.code = NULL ;
            return (c) ;
        } else {
            yylval.code = gen_code (temp_str) ; // aunque no se use
            return (symbol->token) ;
        }
    }

//    printf ("\nDEV: LITERAL %d #%c#\n", (int) c, c) ;        // PARA DEPURAR
    if (c == EOF || c == 255 || c == 26) {
//        printf ("tEOF ") ;        // PARA DEPURAR
        return (0) ;
    }

    return c ;
}

int main ()
{
    yyparse () ;
    return 0 ;
}
```